

COL100: Introduction to Computer Science

Lab 6: Loops and Lists

February 10, 2024

1 Loops

Loops are essential constructs that allow you to repeat a block of code multiple times. They are used when you want to perform a certain task repeatedly, such as processing items in a list, performing calculations, or displaying patterns.

In this tutorial, we will cover two types of loops: the `while` loop and the `for` loop.

1.1 The while Loop

The `while` loop repeatedly executes a block of code as long as a specified condition is true.

Syntax

```
1 while condition:
2     # Code block to be repeated
```

Different components in the structure of the `while` loop are as follows.

1. **condition:** This statement is used to specify the condition that determines whether the loop should continue executing or not. This condition is checked before the execution of the loop body in every iteration. If the condition is true then the loop body is executed; otherwise, if the condition is false, the loop is terminated.
2. **loop body:** This includes the block of code that needs to be executed repetitively. It is indented under the `while` statement.

After the loop termination, execution of the program continues from the line after the loop body.

Example

Printing even numbers from 2 to 10 using a `while` loop:

```
1 num = 2
2 while num <= 10:
3     print(num, end=' ')
4     num += 2
```

In this example,

1. `num = 2` initializes the variable `num` to the value 2.
2. `num <= 10` checks whether the value of `num` is less than or equal to 10. As long as this condition is true, the loop will continue executing.
3. The code inside the loop will be executed as long as the condition is true.
4. `num += 2` increments the value of `num` by 2 in each iteration. This step ensures that we're working with even numbers.

The loop will execute as follows:

```
num is 2. Condition is true. Print 2. Increment num to 4.
num is 4. Condition is true. Print 4. Increment num to 6.
num is 6. Condition is true. Print 6. Increment num to 8.
num is 8. Condition is true. Print 8. Increment num to 10.
num is 10. Condition is true. Print 10. Increment num to 12.
num is 12. Condition is false. Loop terminates.
Output: 2 4 6 8 10
```

1.2 The for Loop

The for loop is a versatile loop structure that allows you to iterate over a sequence.

Syntax

```
1 for variable in sequence:
2     # Code block to be repeated
```

Different components in the structure of `for` loop are

1. **variable:** This variable takes on the value of each element in the sequence during each iteration of the loop.
2. **sequence:** This is the collection of items that the loop iterates over (e.g., a list, string, or range).
3. **loop body:** This includes the block of code that needs to be executed repetitively. It is indented under the `for` statement.

Example

Let's say we want to print numbers from 1 to 5 using a `for` loop:

```
1 for i in range(1, 6):  
2     print(i, end=' ')
```

In this example,

1. `for i in range(1, 6):` initializes the loop control variable `i` to values from 1 to 5.
2. The loop will continue executing until all values in the specified range have been processed.

The loop will execute as follows:

```
i is 1. Print 1.  
i is 2. Print 2.  
i is 3. Print 3.  
i is 4. Print 4.  
i is 5. Print 5.  
Output:  1 2 3 4 5
```

1.3 Nested Loops

Nested loops involve placing one loop inside another, typically used to perform repetitive actions within another set of repetitive actions.

Syntax

```
1 for variable1 in sequence1:  
2     for variable2 in sequence2:  
3         # code block of inside loop  
4     # code block of outer loop
```

The inner loop is executed in every iteration of the outer loop. Nested loops can be a combination of `for` and `while` loops.

Example

Here, nested loops are used to print the table of numbers from 1 to `n`.

```
1 n = 4  
2  
3 for i in range(1, n + 1):  
4     print(f"Table of {i}")  
5
```

```

6     for j in range(1, 11):
7         print(i * j, end=' ')
8
9     print()

```

Output:

Table of 1

1 2 3 4 5 6 7 8 9 10

Table of 2

2 4 6 8 10 12 14 16 18 20

Table of 3

3 6 9 12 15 18 21 24 27 30

Table of 4

4 8 12 16 20 24 28 32 36 40

Try: Can you tell how many times the inner loop executes in the execution of the above example program?

1.4 break Statement

The **break** statement is used to prematurely exit from a loop. It can be used in both simple and nested loops.

Example

Here in this example, we find the first integer in the given range [**start**, **end**] which is divisible by **n**.

```

1 start = 45
2 end = 62
3 n = 13
4
5 for i in range(start, end + 1):
6     if i % n == 0:
7         print(i)
8         break

```

Output: 52

1.5 continue Statement

The **continue** statement skips the rest of the current iteration's statements and moves to the next iteration.

Example

Here in this example, we print all the integers in the given range `[start, end]` which are not divisible by `n`.

```
1 start = 12
2 end = 29
3 n = 3
4
5 for i in range(start, end + 1):
6     if i % n == 0:
7         continue
8     print(i, end=' ')
9 print()
```

Output: 13 14 16 17 19 20 22 23 25 26 28 29

2 Submission Problem

Write a program that generates a pattern based on the input value **n**. The pattern should consist of letters from the English alphabet in an incremental manner. Each line of the pattern should have the same letter repeated a certain number of times.

Requirements

1. Your program should take a positive integer **n** ($1 \leq n \leq 26$) as input from the user.
2. Implement a loop structure to generate the pattern based on the value of **n**.
3. Each line of the pattern should consist of a specific letter repeated a certain number of times.
4. NOTE: There is a space after each character.

Sample Input

```
1 4
```

Sample Output

```
1 A
2 B B
3 C C C
4 D D D D
```

Sample Input

```
1 2
```

Sample Output

```
1 A
2 B B
```

3 Practice Problems

3.1 Finding Maximum and Minimum

Write a program to find the maximum and minimum elements. You are given the number of elements N as the first input, followed by N lines each containing an integer. The program should output two integers, a maximum and a minimum.

Requirements

1. Input:

- The first line contains an integer n ($1 \leq N \leq 100$), which represents the number of elements to be fed as inputs.
- The next N lines contains one integer each.

2. Output:

- Print the maximum value in the first line.
- Print the minimum value in the second line.

Sample Input

```
1 5
2 10
3 3
4 8
5 15
6 6
```

Sample Output

```
1 15
2 3
```

Sample Input

```
1 6
2 -1
3 22
4 14
5 0
6 5
7 13
```

Sample Output

```
1 22
2 -1
```

3.2 Prime Number Checker

Write a program that checks whether a given positive integer is a prime number or not. A prime number is a positive integer greater than 1 that has no positive divisors other than 1 and itself.

Requirements

1. Your program should take a single positive integer as input from the user.
2. Implement a loop structure to check for divisors of the input number.
3. Print YES or NO indicating whether the input number is prime or not.
4. Check for INVALID INPUT.

Sample Input

```
1 7
```

Sample Output

```
1 YES
```

Sample Input

```
1 12
```

Sample Output

```
1 NO
```

3.3 Palindrome Number Checker

Write a Python program that checks whether a given positive integer is a palindrome or not. A palindrome number is a number that reads the same forward and backward.

Requirements

1. Your program should take a positive integer as input from the user.
2. Implement a loop structure to find the reverse number and check if the given number is a palindrome.

3. Display an appropriate message indicating whether the input number is a palindrome or not.

Sample Input

```
1 121
```

Sample Output

```
1 121 is a palindrome number.
```

Sample Input

```
1 12345
```

Sample Output

```
1 12345 is not a palindrome number.
```